# PC-2025-2026 Course Project: Parallel N-gram Frequency Analysis with OpenMP

Yassin Belmaati
yassin.belmaati@edu.unifi.it

## Abstract

*This report presents the design, implementation, and evaluation of a parallel n-gram frequency counting system built with OpenMP. The system computes histograms of character and word bigrams and trigrams over large text corpora. A sequential baseline is compared against a parallel implementation that exploits thread-local hash maps to avoid synchronization overhead in the critical counting loop. Experiments were conducted on a system equipped with an 11th Gen Intel Core i7-11800H CPU, featuring 8 physical cores and 16 logical threads (with Hyper-Threading), and 16 GB of RAM. The evaluation considers corpora of varying sizes and explores scalability across different thread counts, OpenMP scheduling strategies (`static`, `dynamic`, `guided`), and chunk sizes.*

## Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

## 1. Introduction

N-grams are sequences of $n$ consecutive tokens in a text and are widely used in natural language processing. In particular, bigrams ($n$=2) and trigrams ($n$=3), both at word and character level, are common in tasks such as language modeling, spell checking, and data compression.

Counting the frequency of all n-grams in a large corpus can be expensive. For a corpus with $N$ tokens, a simple approach requires $\mathcal{O}(N)$ insertions into a hash map, each involving string creation and lookup. With large datasets, this can take several seconds, making parallel solutions useful.

The goals of this project are:

- Implement sequential and parallel versions of bigram and trigram counting (words and characters) in C++17 with OpenMP.
- Measure speedup with different numbers of threads (1, 2, 4, 8, 16) and corpus sizes.
- Compare different OpenMP scheduling strategies.
- Analyze scalability limitations.

## 2. Sequential Algorithm

### 2.1. Pipeline Overview

The sequential pipeline consists of three stages; only the third is included in timing measurements.

**Stage 1: Normalization.** The raw text is scanned character by character. Alphabetic characters are lowercased; whitespace sequences are collapsed to a single space; punctuation and digits are dropped. The result is a clean ASCII string over the alphabet `a--z` plus the space separator.

**Stage 2: Tokenization.** Two token streams are derived from the normalized text: a word sequence (split on spaces) stored as `std::vector<string>`, and a character sequence (all `a--z` characters, spaces excluded) stored as `std::vector<char>`.

**Stage 3: N-gram counting.** Four counting functions iterate over the respective token vectors and accumulate frequencies into an `unordered_map<string, long long>`.

### 2.2. Data Structure

We use `std::unordered_map` because it offers average-case $\mathcal{O}(1)$ insert and lookup operations, and is simple and portable.

For character n-grams, the sequential version uses a fixed-length `std::string` (2 or 3 characters) reused across iterations to avoid repeated allocations. In the parallel version the key is declared inside the loop body to eliminate any sharing between threads.

For word n-grams, instead, we must build the key by concatenating words (e.g., `w[i] + " " + w[i+1]`), which introduces a higher cost per iteration due to memory allocation.

### 2.3. Pseudocode

Algorithm 1 describes the sequential counting procedure for the general case; the four concrete functions (word/char $\times$ bigram/trigram) are instantiations of this template.

**Algorithm 1** Sequential N-gram Count

**Require:** token vector $T$ of length $N$, n-gram order $n$
**Ensure:** frequency map $M$

```
for i ← 0 to N − n do
    k ← BuildKey(T[i], ..., T[i + n − 1])
    M[k] += 1
end for
return M
```

BUILDKEY concatenates the $n$ tokens with a space separator for word n-grams, or writes $n$ characters into a pre-allocated buffer for character n-grams. The loop has no cross-iteration data dependency on $T$; the only shared state is the map $M$, which is the target of parallelization.

## 3. Parallel Algorithm

### 3.1. Synchronization Strategy

Three synchronization approaches were considered for concurrent map updates:

1. **Global map + critical section.** Each thread acquires a lock before every insertion. This serializes all updates and in practice performs worse than the sequential baseline at high thread counts.
2. **Atomic operations.** Not applicable to `std::unordered_map`, which provides no atomic interface.
3. **Thread-local maps + sequential merge.** Each thread maintains a private WORDMAP. After the parallel region a single sequential pass merges all local maps into the global result. This eliminates all synchronization inside the hot loop.

Option 3 was adopted because it avoids synchronization during the main counting loop. Each thread works independently on its own local map, so there is no interference or waiting between threads while processing the data.

Once the parallel phase is complete, the local maps are combined in a final step to produce the global result. This approach keeps the parallel part simple and efficient, and limits coordination to a single post-processing phase.

### 3.2. OpenMP Implementation

Algorithm 2 describes the parallel counting procedure.

**Algorithm 2** Parallel N-gram Count (OpenMP)

**Require:** token vector $T$, n-gram order $n$, thread count $P$, schedule kind $s$, chunk size $c$
**Ensure:** frequency map $M_{\text{global}}$

```
omp_set_schedule(s, c)
allocate {M_0, ..., M_{P−1}} {one map per thread} i ← 0
    to N − n schedule(runtime)
k ← BuildKey(T[i], ..., T[i + n − 1])
M_tid[k] += 1
for p ← 0 to P − 1 do
    {sequential merge}
    for each (k, v) ∈ M_p do
        M_global[k] += v
    end for
end for
return M_global
```

The input tokens are processed in parallel using an OpenMP loop. Each thread builds n-gram keys from consecutive tokens and updates its own local map $M_{\texttt{tid}}$, avoiding any interaction with other threads during this phase.

After all iterations are completed, the local maps $M_0, \ldots, M_{P-1}$ are combined in a final step. This merge is done sequentially by iterating over each local map and accumulating the counts into the global map $M_{\text{global}}$.

## 4. Tests and Results

### 4.1. Experimental Setup

**Hardware.**

| | |
|---|---|
| CPU | `11th Gen Intel(R) Core(TM) i7-11800H` |
| Physical cores | `8` |
| Logical threads | `16` |

**Software.**
Compiler: `GCC 13.3`
flags: `-O3 -march=native -fopenmp -std=c++17`
OS: `Ubuntu 24.04`.

**Dataset.** A plain-text corpus from Project Gutenberg was repeated $k$ times to produce four corpus sizes corresponding to $k \in \{100, 200, 500, 800\}$ repetitions of the base file.

**Methodology.** Each configuration was executed 7 times; the first run was discarded as warm-up, leaving 6 measurements per configuration. Wall-clock time was measured with `omp_get_wtime()` bracketing only the n-gram counting phase. Reported speedup uses the best (minimum) sequential time measured within the same execution run as the parallel version (`mode=both`), ensuring both times are obtained under identical system conditions:

$$S(p) = \frac{T_{\text{seq}}^{\min}}{T_{\text{par}}^{\min}(p)}. \tag{1}$$

### 4.2. Correctness Validation

Correctness was verified by running both sequential and parallel versions on the same input and comparing all four output maps entry by entry. All parallel configurations produced results identical to the sequential baseline across all tested inputs, confirming that the thread-local + merge strategy is race-free and produces no lost updates.

### 4.3. Sequential vs. Parallel Time

Figure 1 shows the mean wall-clock time of the sequential baseline and of the best parallel configuration as a function of corpus size. The gap between the two curves increases with the input size: for larger corpora, the fixed overhead of thread management and the final merge is amortized over more work, making the parallel implementation progressively more advantageous.

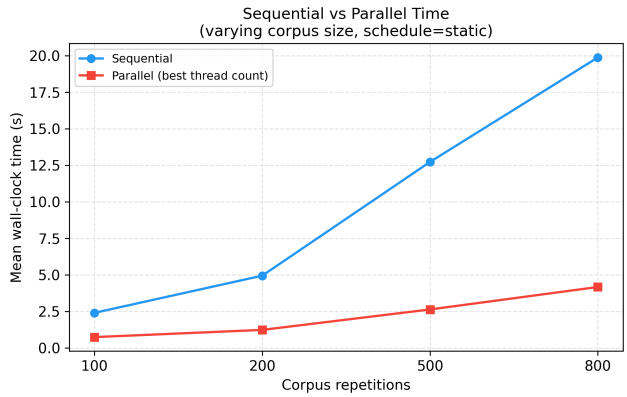


Figure 1. Sequential vs. parallel mean time as corpus size increases (best thread count per size).

Figure 2 reports the mean execution time as a function of thread count for a fixed corpus size ($k$=500). The parallel version improves steadily up to 16 threads, with diminishing returns beyond 8 threads, consistent with the 8 physical cores available on the test machine and with the fact that the workload is dominated by hash-map updates and string handling rather than by purely compute-bound operations.

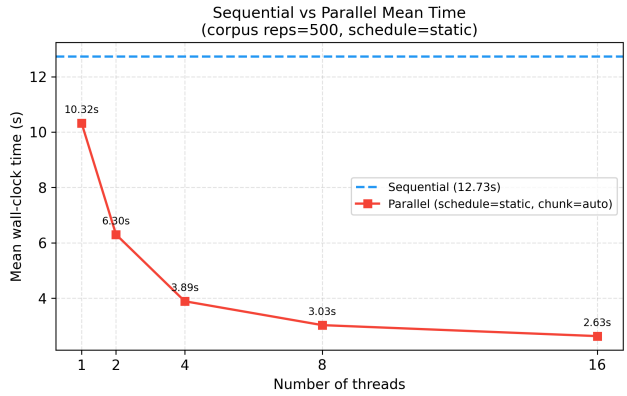


Figure 2. Sequential (dashed) vs. parallel mean time as thread count increases ($k$=500).

### 4.4. Speedup and Efficiency

Table 1 summarizes the best observed thread count for each corpus size together with the cor-

responding speedup and parallel efficiency. The optimal configuration varies with corpus size: for the smallest corpus ($k$=100), 8 threads yield the best result and adding a 16th thread slightly degrades performance due to Hyper-Threading overhead exceeding the parallelization benefit on this memory-bound workload. For larger corpora, 16 threads remain advantageous as the additional work is sufficient to amortize the extra scheduling cost.

| Corpus size | Best threads | Speedup $S(p)$ | Efficiency (%) |
|---|---|---|---|
| 100 | 8 | 3.12 | 39.00 |
| 200 | 16 | 3.62 | 22.63 |
| 500 | 16 | 4.38 | 27.38 |
| 800 | 16 | 4.78 | 29.88 |

Table 1. Best observed configuration for each corpus size.

Figure 3 shows the speedup curves across thread counts. The trends are consistent with the hardware characteristics of the test machine: all curves show clear gains from 2 to 8 threads, while the transition to 16 threads yields smaller and corpus-dependent returns. For $k$=100, speedup at 16 threads is marginally lower than at 8, reflecting the cost of Hyper-Threading on a workload dominated by irregular memory accesses and hash-map updates rather than compute-bound operations. For larger corpora the curves remain monotonically increasing up to 16 threads, with the largest absolute speedup observed at $k$=800, where the parallelizable fraction of the workload is most effectively exploited.

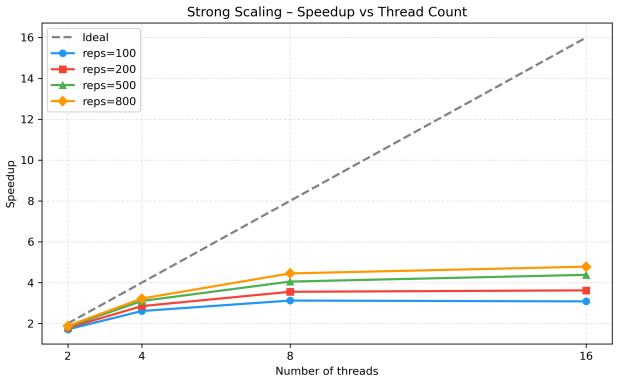


Figure 3. Speedup curves for all corpus sizes.

Parallel efficiency decreases monotonically as the number of threads increases, as expected from Amdahl's law: the serial merge phase constitutes a fixed overhead that becomes proportionally more costly at higher thread counts. Efficiency is highest at 2 threads across all corpus sizes, and remains reasonable (above 30%) up to 8 threads for the larger corpora. Beyond 8 threads, efficiency drops more sharply, particularly for small corpora where the parallelizable work per thread is limited.

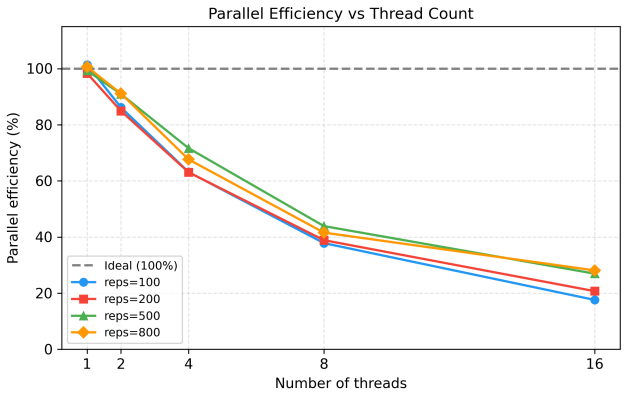


Figure 4. Parallel efficiency vs. thread count for all corpus sizes.

### 4.5. Scheduling Strategy Comparison

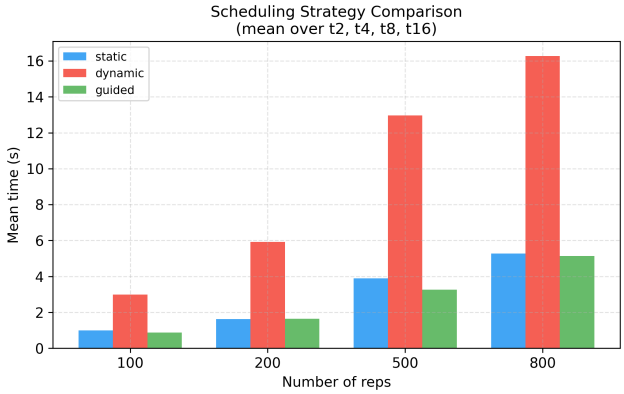


Figure 5. Mean time for `static`, `dynamic`, and `guided` scheduling across corpus sizes (mean over $t \in \{2, 4, 8, 16\}$).

Figure 5 compares the OpenMP scheduling strategies across corpus sizes and chunk sizes (t $\in \{2, 4, 8, 16\}$, chunk $\in \{64, 256\}$).

In general, static scheduling is a strong default choice because the per-iteration work is fairly regular: every iteration performs a similar hash-

map update on a token window of fixed size, so there is little benefit from dynamic load balancing. guided behaves similarly to static across most configurations, as the decreasing chunk size it uses converges quickly to a regime comparable to a static partition. dynamic with chunk=0, which OpenMP internally treats as chunk=1, introduces prohibitive scheduling overhead: each thread fetches a single iteration at a time from a shared queue, resulting in times up to 4× higher than static even at t=1. Explicitly setting a chunk size greater than 64 recovers performance comparable to guided. Overall, the results confirm that the workload is sufficiently uniform for static scheduling to remain optimal, while flexible strategies only matter when runtime variability or load imbalance becomes significant

### 4.6. Chunk Size Effect

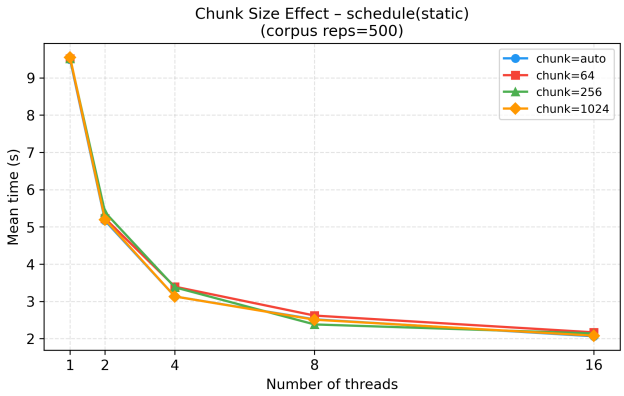


Figure 6. Effect of chunk size on mean parallel time for `static` ($k$=500).

Figures 6 and 7 show the effect of chunk size on mean parallel time for `static` and `dynamic` respectively ($k$=500).

For `static`, chunk size has a modest effect: all explicit values (64, 256, 1024) perform similarly, with differences well within measurement noise. The `chunk=0` (auto) case is also competitive, since OpenMP divides iterations evenly by default.

For `dynamic`, the effect is far more pronounced. `chunk=0` (equivalent to `chunk=1`) is dramatically slower due to the per-iteration synchronization overhead discussed in Section 4.5.

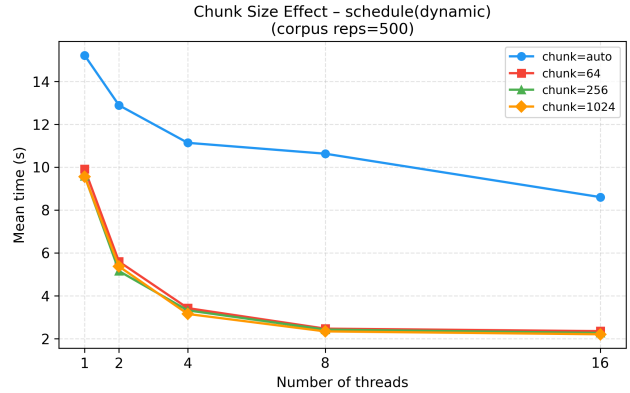


Figure 7. Effect of chunk size on mean parallel time for `dynamic` ($k$=500).

Larger chunk sizes (64–1024) recover performance comparable to `static`, with diminishing differences as chunk size grows. No single chunk size is universally optimal, but values in the range 256–1024 represent a robust choice for this workload.

### 4.7. Scalability Discussion

The implementation shows clear strong scaling: for a fixed corpus size, execution time decreases as the number of threads increases, up to the point where runtime overheads and hardware limits start to dominate. However, the speedup is sublinear. The main limiting factors are the final sequential merge of the thread-local hash maps, the cost of constructing string keys, and the overhead of hash-table operations.

The design choice of using one private hash map per thread is effective because it eliminates synchronization inside the hot loop. At the same time, the final merge introduces a serial reduction phase that limits the maximum achievable speedup according to Amdahl's law. This makes the implementation a good example of a map-reduce style strategy on shared-memory hardware.

A further observation is that the problem is not particularly SIMD friendly. The core computation is dominated by irregular hash-table updates and string handling, which limits the effectiveness of explicit vectorization. In practice, the main performance gains come from thread-level paral-

lelism rather than from SIMD instructions.